

# **PetCare Hub**

## **Documentacao Tecnica do Banco de Dados**

Challenge FIAP 2026 — CLYVO VET

Sprint 3 — Mastering Relational and Non-Relational Database

# 1. Explicação da Modelagem

O banco de dados do projeto **PetCare Hub** foi modelado para apoiar a continuidade do cuidado veterinário dentro do contexto do Challenge FIAP 2026 — CLYVO VET. A estrutura foi desenvolvida com foco em organização, integridade dos dados, escalabilidade e integração com as APIs Java Spring Boot e .NET do projeto.

O objetivo principal da modelagem é centralizar informações clínicas, preventivas e operacionais do ecossistema PetCare Hub, permitindo o gerenciamento completo da jornada de saúde do pet. O banco foi projetado para armazenar dados relacionados aos tutores, clínicas veterinárias, pets, consultas, protocolos preventivos, dispositivos IoT, leituras de sensores, alertas de saúde e score de saúde.

A modelagem foi construída seguindo os princípios da modelagem relacional e da 3ª Forma Normal (3FN), reduzindo redundâncias e garantindo consistência dos dados. Além disso, a estrutura foi organizada para permitir consultas analíticas, geração de relatórios, monitoramento contínuo do pet e integração com futuras funcionalidades da plataforma.

O projeto também foi pensado para suportar funcionalidades preventivas e inteligentes, utilizando dados clínicos e leituras de sensores para geração de alertas e acompanhamento longitudinal da saúde animal. Dessa forma, o banco de dados atua como base central para as regras de negócio, APIs, dashboards clínicos e futuras evoluções do ecossistema PetCare Hub.

O banco possui 14 tabelas: **TUTOR**, **CLINICA**, **PET**, **CONSULTA**, **PROTOCOLO\_PREVENTIVO**, **EVENTO\_PREVENTIVO**, **DISPOSITIVO\_IOT**, **LEITURA\_COLEIRA**, **LEITURA\_COMEDOURO**, **LEITURA\_AMBIENTE**, **ALERTA\_SAUDE**, **SCORE\_SAUDE**, **LOG\_ERROS** e **AUDITORIA\_TUTOR** (criada pelo trigger de auditoria). A tabela TUTOR nasceu como **RESPONSAVEL** e foi renomeada para alinhar com a terminologia usada no restante do produto.

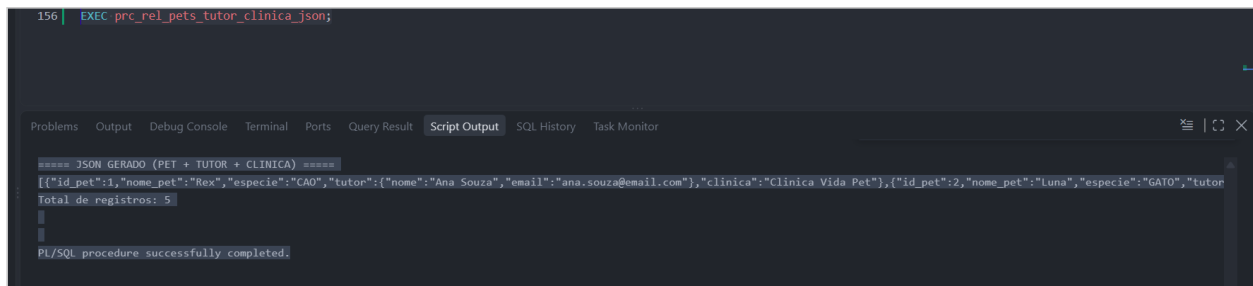
## 2. Desenvolvimento de 02 Procedures

As duas procedures abaixo atendem ao requisito da rubrica oficial da Sprint 3: cada uma trata pelo menos 3 exceções específicas (além de OTHERS). A seguir, prints reais da execução no Oracle SQL Developer, com um caso de sucesso para cada uma e a evidência das exceções registradas na tabela LOG\_ERROS.

### 2.1 PRC\_REL\_PETS\_TUTOR\_CLINICA\_JSON — JOIN + conversão para JSON manual

Faz JOIN entre PET, TUTOR e CLINICA e monta um array JSON linha a linha, chamando a function FN\_PET\_PARA\_JSON (sem usar TO\_JSON/JSON\_OBJECT). Trata 4 exceções distintas.

Caso de sucesso (execução real — 5 pets):



```
156 | EXEC prc_rel_pets_tutor_clinica_json;

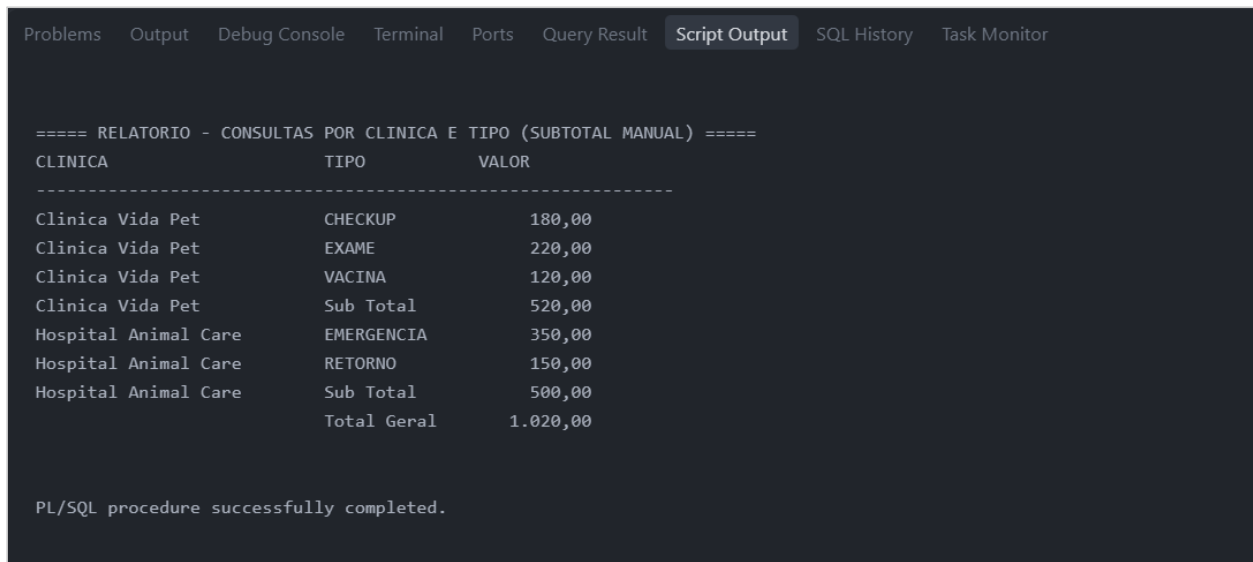
==== JSON GERADO (PET + TUTOR + CLINICA) ====
[{"id_pet":1,"nome_pet":"Rex","especie":"CAO","tutor":{"nome":"Ana Souza","email":"ana.souza@email.com"},"clinica":"Clinica Vida Pet"}, {"id_pet":2,"nome_pet":"Luna","especie":"GATO","tutor":
Total de registros: 5

PL/SQL procedure successfully completed.
```

### 2.2 PRC\_REL\_CONSULTAS\_SUBTOTAL — subtotal e total geral manuais

Lê CONSULTA categorizada por CLINICA (categoria 1) e TIPO\_CONSULTA (categoria 2), agrupando combinações repetidas com SUM(NVL(valor,0)) e calculando subtotal por clínica e total geral manualmente em um loop, sem ROLLUP/CUBE/GROUPING SETS. Trata 4 exceções distintas.

Caso de sucesso (execução real):

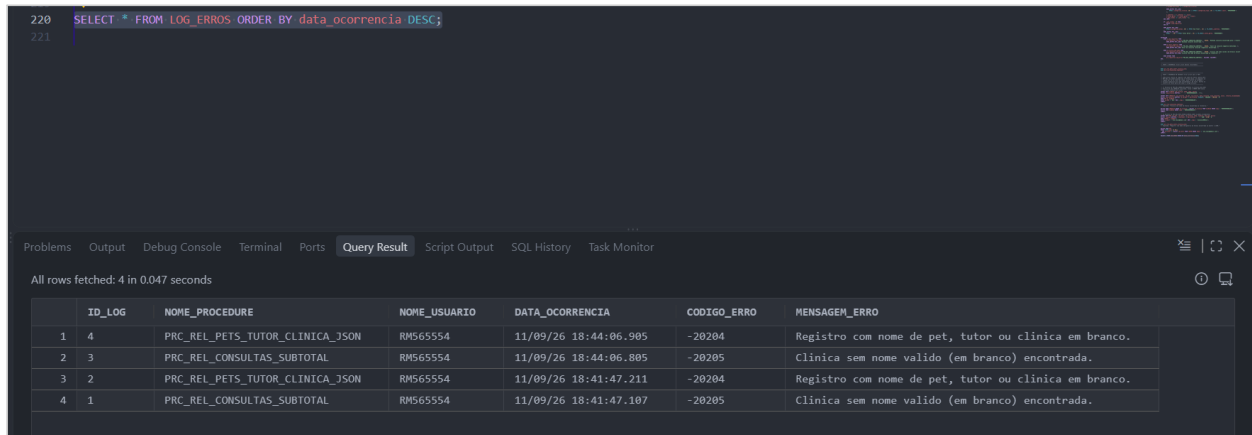


```
==== RELATORIO - CONSULTAS POR CLINICA E TIPO (SUBTOTAL MANUAL) ====
CLINICA          TIPO          VALOR
-----
Clinica Vida Pet  CHECKUP        180,00
Clinica Vida Pet  EXAME          220,00
Clinica Vida Pet  VACINA         120,00
Clinica Vida Pet  Sub Total      520,00
Hospital Animal Care  EMERGENCIA    350,00
Hospital Animal Care  RETORNO        150,00
Hospital Animal Care  Sub Total      500,00
                  Total Geral    1.020,00

PL/SQL procedure successfully completed.
```

## 2.3 Evidência das exceções — tabela LOG\_ERROS

Print real de `SELECT * FROM LOG_ERROS ORDER BY data_ocorrencia DESC;` — confirma que as exceções de nome em branco das duas procedures (e\_clinica\_sem\_nome, e\_dados\_incompletos) foram registradas oficialmente na tabela de log, em duas rodadas de teste distintas.



The screenshot shows a SQL IDE interface. At the top, a query is entered: `SELECT * FROM LOG_ERROS ORDER BY data_ocorrencia DESC;`. Below the query editor, the 'Query Result' tab is active, displaying a table with 4 rows and 7 columns. The columns are: ID\_LOG, NOME\_PROCEDURE, NOME\_USUARIO, DATA\_OCORRENCIA, CODIGO\_ERRO, and MENSAGEM\_ERRO. The data shows four error records from two different procedures, both executed by user RM565554. The messages indicate errors related to missing names for pet, tutor, or clinic.

|   | ID_LOG | NOME_PROCEDURE                  | NOME_USUARIO | DATA_OCORRENCIA       | CODIGO_ERRO | MENSAGEM_ERRO                                         |
|---|--------|---------------------------------|--------------|-----------------------|-------------|-------------------------------------------------------|
| 1 | 4      | PRC_REL_PETS_TUTOR_CLINICA_JSON | RM565554     | 11/09/26 18:44:06.905 | -20204      | Registro com nome de pet, tutor ou clinica em branco. |
| 2 | 3      | PRC_REL_CONSULTAS_SUBTOTAL      | RM565554     | 11/09/26 18:44:06.805 | -20205      | Clinica sem nome valido (em branco) encontrada.       |
| 3 | 2      | PRC_REL_PETS_TUTOR_CLINICA_JSON | RM565554     | 11/09/26 18:41:47.211 | -20204      | Registro com nome de pet, tutor ou clinica em branco. |
| 4 | 1      | PRC_REL_CONSULTAS_SUBTOTAL      | RM565554     | 11/09/26 18:41:47.107 | -20205      | Clinica sem nome valido (em branco) encontrada.       |

## 3. Desenvolvimento de 02 Functions

Além das duas functions de negócio do produto (FN\_CALCULAR\_IDADE\_PET e FN\_SCORE\_MEDIO\_PET, compiladas com sucesso), a rubrica da Sprint 3 exige 2 functions com regras próprias: conversão manual para JSON e validação de senha — ambas com tratamento de exceções nomeadas, demonstradas a seguir com prints reais da execução.

### 3.1 FN\_PET\_PARA\_JSON — conversão relacional → JSON manual

Monta manualmente (concatenação de string) um JSON a partir de PET + TUTOR + CLINICA. Não usa TO\_JSON, JSON\_OBJECT, JSON\_VALUE nem nenhuma função built-in de JSON do Oracle. Escapa manualmente aspas, barra invertida, quebra de linha e tabulação. Trata 4 exceções distintas.

Caso de sucesso:

```
125 SELECT fn_pet_para_json(  
126     p.id_pet, p.nome, p.especie, t.nome, t.email, c.nome  
127 ) AS json_pet  
128 FROM PET p  
129 JOIN TUTOR t ON t.id_tutor = p.id_tutor  
130 JOIN CLINICA c ON c.id_clinica = p.id_clinica  
131 WHERE p.nome = 'Rex';  
132  
133 -- Caso de excecao da Funcao 1 (id pet nulo)  
134 SELECT fn_pet_para_json(NULL, 'Teste', 'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste') AS json_erro  
135 FROM DUAL;  
136  
137 -- Caso de escape manual (barra invertida, aspas, quebra de  
138 -- linha e tabulacao no mesmo valor) - print para a documentacao  
139 SELECT fn_pet_para_json(  
140     99,  
141     'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
142     'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
143 ) AS json_escapa_teste  
144 FROM DUAL;
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.030 seconds

|   | JSON_PET                                                                                                                              |
|---|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | {"id_pet":1,"nome_pet":"Rex","especie":"CAO","tutor":{"nome":"Ana Souza","email":"ana.souza@email.com"},"clinica":"Clinica Vida Pet"} |

Caso de exceção (id\_pet nulo):

```
SELECT fn_pet_para_json(NULL, 'Teste', 'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste') AS json_erro  
FROM DUAL;  
  
-- Caso de escape manual (barra invertida, aspas, quebra de  
-- linha e tabulacao no mesmo valor) - print para a documentacao  
SELECT fn_pet_para_json(  
    99,  
    'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
    'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
)
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.014 seconds

|   | JSON_ERRO                                     |
|---|-----------------------------------------------|
| 1 | {"erro":"id_pet obrigatorio para gerar JSON"} |

Caso de escape manual (aspas, barra invertida, quebra de linha e tabulação no mesmo valor):

```
139 SELECT fn_pet_para_json(  
140     99,  
141     'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
142     'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
143 ) AS json_escapa_teste  
144 FROM DUAL;  
145  
146 -- Caso de sucesso da Funcao 2  
147 SELECT fn_validar_forca_senha('Senha123') AS resultado FROM DUAL;
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.015 seconds

|   | JSON_ESCAPE_TESTE                                                                                                                                                              |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | {"id_pet":99,"nome_pet":"Nome com \aspas\ e barra \ contra\ quebra de linha\ttab","especie":"CAO","tutor":{"nome":"Tutor Teste","email":"teste@teste.com"},"clinica":"Cli..."} |

## 3.2 FN\_VALIDAR\_FORCA\_SENHA — validação de senha do fluxo de ativação do TUTOR

Valida a senha usada na ativação de conta do TUTOR (mínimo 8 caracteres, com letra e número). Trata 4 exceções distintas.

Caso de sucesso ('Senha123'):

```
146 -- Caso de sucesso da Funcao 2
147 SELECT fn_validar_forca_senha('Senha123') AS resultado FROM DUAL;
148
149 -- Casos de excecao da Funcao 2 (print para a documentacao)
150 SELECT fn_validar_forca_senha(NULL) AS resultado FROM DUAL;
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.073 seconds

|   | RESULTADO |
|---|-----------|
| 1 | VALIDA    |

Caso de exceção — senha nula:

```
150 SELECT fn_validar_forca_senha(NULL) AS resultado FROM DUAL;
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.015 seconds

|   | RESULTADO                      |
|---|--------------------------------|
| 1 | INVALIDA: senha nao informada. |

Caso de exceção — senha curta ('abc'):

```
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.016 seconds

|   | RESULTADO                                        |
|---|--------------------------------------------------|
| 1 | INVALIDA: senha deve ter no minimo 8 caracteres. |

Caso de exceção — senha sem número ('semnumero'):

```
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.013 seconds

|   | RESULTADO                                       |
|---|-------------------------------------------------|
| 1 | INVALIDA: senha deve conter ao menos um numero. |

## 4. Desenvolvimento de 01 Trigger para Auditoria

O trigger TRG\_AUDITORIA\_TUTOR dispara AFTER INSERT OR UPDATE OR DELETE ON TUTOR, FOR EACH ROW, e grava em AUDITORIA\_TUTOR a operação, o usuário do banco (USER), a data/hora (SYSTIMESTAMP) e o status\_acesso/email antes e depois da mudança. A tabela AUDITORIA\_TUTOR, a sequence SEQ\_AUDITORIA\_TUTOR e o trigger foram criados com sucesso, e a demonstração ponta a ponta abaixo executou um INSERT, um UPDATE (PRE\_CADASTRADO → ATIVO) e um DELETE em um tutor de teste.

Print real de SELECT \* FROM AUDITORIA\_TUTOR (filtrado pelo tutor de teste):

```
96 SELECT id_auditoria, id_tutor, operacao, usuario_bd, data_hora,
97        status_anterior, status_novo, email_anterior, email_novo
98 FROM AUDITORIA_TUTOR
99 WHERE email_anterior = 'auditoria@teste.com' OR email_novo = 'auditoria@teste.com'
100 ORDER BY data_hora;
101 -- Esperado: 3 linhas (INSERT, UPDATE, DELETE) para esse tutor,
102 -- com status_anterior/status_novo mostrando PRE_CADASTRADO -> ATIVO
103 -- no UPDATE, e id_tutor/email preenchidos em todas as 3 linhas.
```

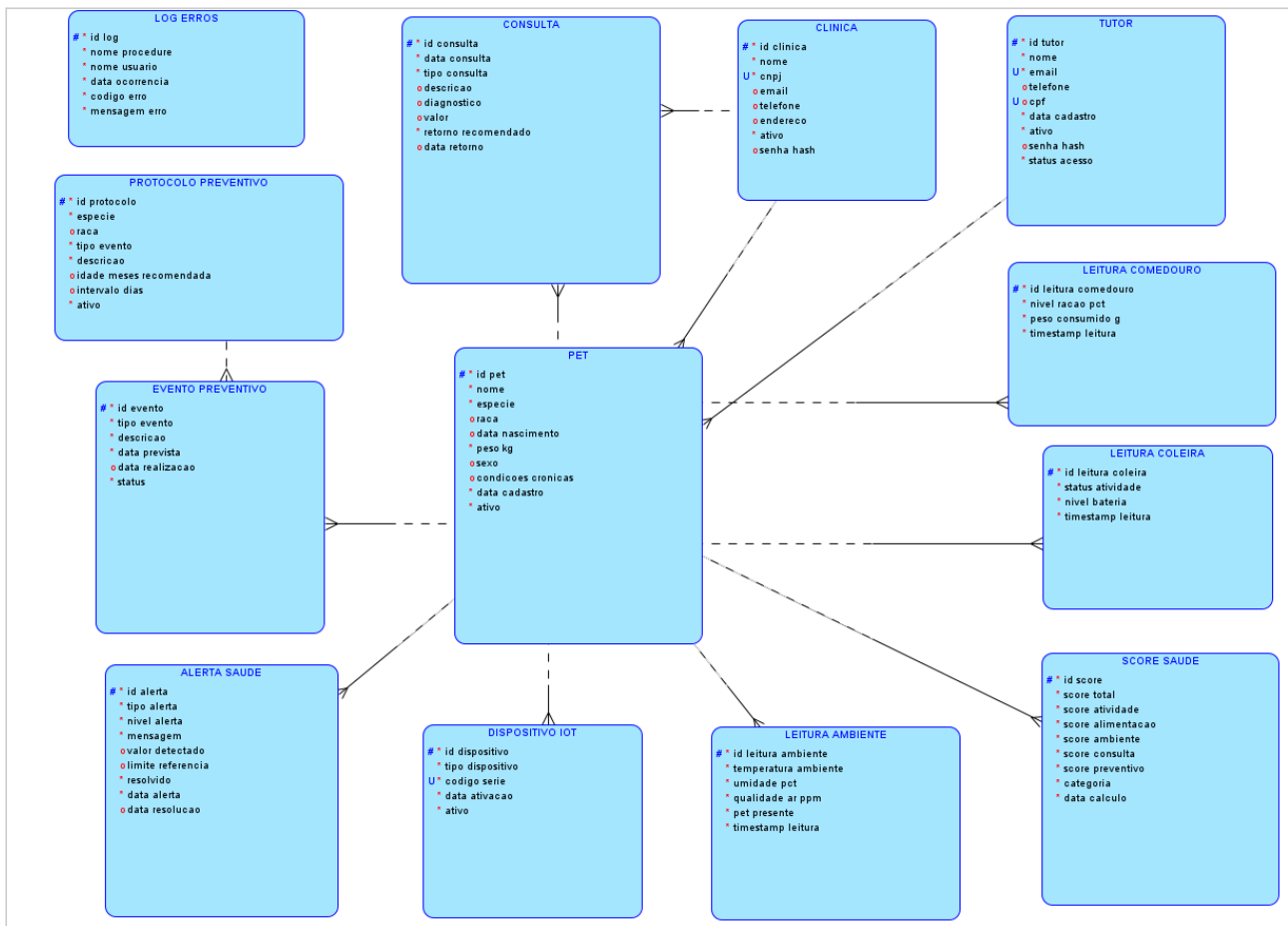
Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 3 in 0.036 seconds

|   | ID_AUDITORIA | ID_TUTOR | OPERACAO | USUARIO_BD | DATA_HORA             | STATUS_ANTERIOR | STATUS_NOVO    | EMAIL_ANTERIOR      | EMAIL_NOVO          |
|---|--------------|----------|----------|------------|-----------------------|-----------------|----------------|---------------------|---------------------|
| 1 | 1            | 7        | INSERT   | RM565554   | 11/09/26 18:47:54.489 | (null)          | PRE_CADASTRADO | (null)              | auditoria@teste.com |
| 2 | 2            | 7        | UPDATE   | RM565554   | 11/09/26 18:47:54.522 | PRE_CADASTRADO  | ATIVO          | auditoria@teste.com | auditoria@teste.com |
| 3 | 3            | 7        | DELETE   | RM565554   | 11/09/26 18:47:54.549 | ATIVO           | (null)         | auditoria@teste.com | (null)              |

As 3 linhas confirmam a auditoria completa: quem alterou (usuário do banco), quando (data/hora com milissegundos) e o estado antes/depois de cada operação — com destaque para a transição PRE\_CADASTRADO → ATIVO capturada no UPDATE.

## 5. DER



## 6. MER

